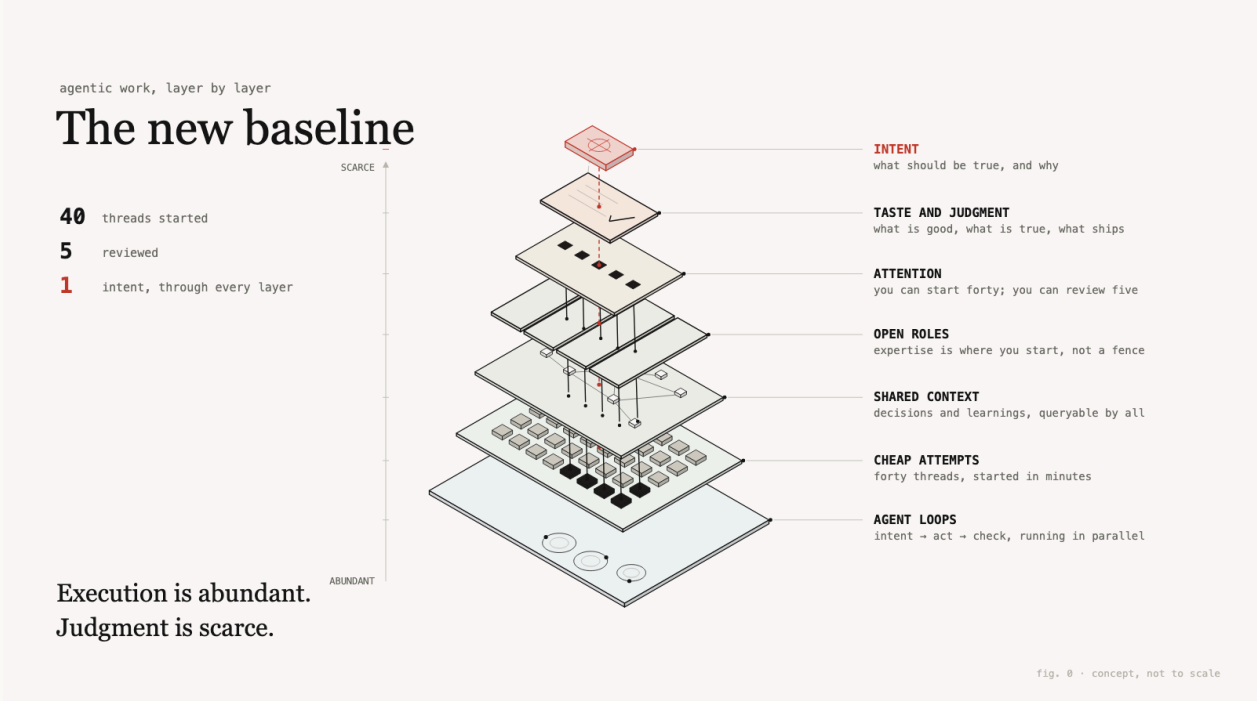
A WORKING PLAYBOOK

The New Baseline

Agentic work and agentic leadership: contracts, checks, rituals and templates.



Carlos D. Escobar-Valbuena · broomva.tech

```
npx skills add broomva/skills --skill agentic-leadership
```

The New Baseline

A working playbook for agentic work and agentic leadership.

This is the practical companion to the essay *The New Dynamics of Agentic Leadership*. The essay argues why work is changing. This playbook is how to work inside the change: the contracts, the checks, the rituals and the templates.

It is written for two readers:

- **People** who lead work, which now means almost everyone, because anyone who directs agents is leading something.
- **Agents**, through the `agentic-leadership` skill, so the agents you work with hold the same standards you do. Install it with `npx skills add broomva/skills --skill agentic-leadership`.

Nothing here is specific to code. Any work that can be stated as an intent, delegated and verified is in scope: research, design, operations, writing, analysis, sales preparation, hiring, finance.

---

1. The baseline

The unit of work changed. It used to be a **task**: something one person did in hours or days, once someone else decided it was worth doing. With agents it is becoming a **loop**: state an intent, let agents act, look at what came back, sharpen the intent, run again. A loop can run overnight, next to nine others, in a domain you have never worked in.

When the cost of an attempt drops from days to minutes, three things change:

WHAT CHANGED	BEFORE	NOW
The economics of creation	Plan carefully, attempt rarely	Attempt widely, choose carefully
The boundaries of a role	"That's not my area"	Expertise is where you start
The meaning of not knowing	Wait for an answer	Open a session and find out

The scarce resources are no longer hands. They are **attention, taste and verification**: deciding which of forty possible starts deserve a start, which of ten results deserve trust, and which of two survivors deserve your name.

The five standards

Hold yourself, your team and your agents to these:

1. **Every question is something you can start on today.**

2. **Every repeated task is something you can automate or improve.**
3. **Every piece of work leaves the system better:** a note, a skill, a check.
4. **Freed time is reinvested on purpose:** quality, learning, the workspace, rest.
5. **Nothing here is limited to code.** If it can be stated, delegated and verified, it is in scope.

---

2. Everyone signs the contract twice

Autonomy is a contract. With agents, the contract repeats at every level:

- **Downward**, you are the leader. You set intent for your agents, bound them, review their work and own what ships.
- **Upward**, you are the one delegated to. You receive intent, own a problem and escalate what is not yours to decide.

The same five terms hold at every level of the recursion:

TERM	LEADER → PERSON	PERSON → AGENT FLEET
Intent	the outcome and why it matters	the goal, and what "good" means
Bounds	decision rights, budget, what is off-limits	permissions, budget, gates, what is irreversible
Evidence	what shows the outcome holds	tests, receipts, verifiable outputs
Escalation	what comes back up, and when	when the agent must stop and ask
Review	who judges, on what basis	you, or a different model, before anything ships

A missing term fails the same way at every level. Vague intent produces a busy fleet doing the wrong thing. Missing bounds produce confident damage. Missing evidence produces claims instead of results.

---

3. Write the contract

Write the contract before the work starts, at whatever size the work deserves. A one-hour task gets five lines. A two-week initiative gets a page.

```
## Contract: <short name>

Intent:     <the outcome that should be true> because <why it matters>.
Not in scope: <what this is explicitly not about>.
Bounds:     may <actions allowed without asking>;
            may take costly actions only inside <an explicit budget>;
            must get explicit approval before <irreversible or external actions>.
Evidence:   done means <observable check>, shown by <receipt>.
Escalation: stop and ask when <condition>; otherwise decide and note why.
Review:     <who> reviews <what>, before <ship / merge / send>.
Owner:      <one name>.
```

Example: outside code

```
## Contract: competitor pricing scan

Intent:     we know how the five closest competitors price their entry tier,
            because we are setting ours next week.
Not in scope: positioning advice; anything behind a login.
Bounds:     may read public pages and archived snapshots; must ask before
            contacting any company; budget two hours of agent time.
Evidence:   done means a table with plan, price, limits and a dated source URL
            per cell; any cell without a source is marked unknown.
Escalation: stop and ask if a competitor has no public pricing.
Review:     I read every row against its source before sharing it.
Owner:      me.
```

Example: code

```
## Contract: export to CSV

Intent:     users can export the report table to CSV, because support gets
            five requests a week for it.
Not in scope: Excel formats, scheduled exports.
Bounds:     may change the report module and its tests; must ask before
            touching auth or the database schema.
Evidence:   done means the existing tests pass, a new test covers quoting and
            unicode, and a recorded run downloads a real file.
Escalation: stop and ask if the table exceeds 50,000 rows in production data.
Review:     a different model reviews the diff; I approve the merge.
Owner:      me.
```

---

4. Write intent that survives delegation

Be sharp on the what and the why. Be open on the how and the who.

WEAK INTENT	STRONG INTENT
"Improve onboarding."	"New users reach their first saved report within ten minutes, because most churn happens before that."
"Look into this bug."	"Checkout never fails silently: every failure shows the user a reason and logs a trace."
"Be proactive."	"You own the reliability of the import pipeline: you decide fixes under one day of work, and propose anything larger."

A checklist for every intent:

- ☐ Names an outcome that can be observed, not an activity.
- ☐ Says why, so the person or agent can make trade-offs you did not foresee.
- ☐ Says what is out of scope.
- ☐ Leaves the method open unless the method is the point.
- ☐ Could be judged done or not done by someone other than you.

"Be proactive" is itself a direction, and it needs to be written down: what you own, what you can decide without asking, and what counts as done.

---

5. Bound the risk

Sort actions by how hard they are to undo, and let autonomy follow:

CLASS	EXAMPLES	DEFAULT
Reversible	drafts, research, branches, local experiments	act freely, note what you did
Costly	long compute, many iterations	act inside an explicit budget, report after
Irreversible or external	spending money, messaging a group, publishing, deleting data, merging to production, contacting customers	ask first, every time

Rules that keep bounds real:

- Put bounds where they are enforced, not only where they are written. A rule an agent can ignore is a wish. Gates, permissions and checks are the bounds.
- Give every autonomous run a budget: iterations, time, money and a stop condition for "no progress". If none is stated, do not infer one: prepare reversible work or ask for it.
- Keep a kill switch, and know where it is before you need it.

---

6. Set the evidence bar

"Done" is a claim. Evidence is what lets someone else believe it.

WORK TYPE	EVIDENCE THAT IT IS DONE
Code	tests pass, the changed behavior was exercised, a reviewer who is not the author agreed
Research	every claim has a source that was actually opened; confidence is marked
Design	the artifact was viewed at its real size, on its real surface
Writing	read end to end once by someone other than the author, facts checked
Operations	the system was observed in the new state, not assumed
Data	the numbers reconcile with an independent count

Three rules:

1. **Reasoning is not evidence.** "It should work" is a hypothesis. Run it.
2. **The author does not certify their own work.** For people that means a reviewer. For agents it means a separate session, ideally a different model.
3. **Report what did not run.** An unverified step stated plainly is worth more than a verified-sounding summary.

---

7. Escalate well

Asks are expensive: they spend someone else's attention. A good ask:

- carries its own context: what is happening, what you already checked, links;
- offers options with trade-offs and a recommendation;
- asks for one decision, not a review of everything;
- says what happens by default if nobody answers, and when. The default must be something you are already allowed to do: silence never approves an irreversible or external action. If the question only affects reversible work, the default is to proceed with your recommendation and say so; wait only when every option is costly, irreversible or external.

```
## Ask: <one-line decision>

Context: <two or three sentences, with links>.
Options:
  A. <option>: <trade-off>.  (recommended, because <reason>)
  B. <option>: <trade-off>.
Default: if no answer by <time>, I do A (only if A is within my bounds and
         already authorized, within an explicit budget, and neither irreversible
         nor external; otherwise I wait and report). Silence never authorizes
         costly work outside its budget, or irreversible or external work.
```

Agents escalate on the conditions in their contract and on anything irreversible. Everything else they decide, and write down why.

---

8. Ration attention, not ambition

You can start forty threads. You cannot review forty.

- **Set a work-in-progress limit equal to what you can actually review.** For most people that is a handful of parallel threads, not dozens.
- **Prefer two verified threads over ten half-checked ones.**
- **Kill work that stopped mattering,** including your own. Keep a kill list.
- **Let the system call you, not the other way round.** The measure of an autonomous setup is not how much it does without you. It is how precisely it calls you: a short, ordered list of decisions that need you, and nothing else.
- **Protect the attention you have.** The same attention that feeds and notifications drain is the attention judgment needs.

And decide explicitly where freed time goes. Otherwise abundance quietly turns into permanent availability.

---

9. Make everything queryable

An agent is exactly as good as the context it can reach. Whatever lives only in someone's head does not exist for it.

Where things land:

KIND	WHERE	MINIMUM STRUCTURE
Decisions	one shared workspace, not chat	what, why, alternatives, owner, date
Specs and plans	next to the work they describe	intent, scope, acceptance checks
Research and learnings	a knowledge graph of linked notes	claim, sources, confidence, links
Standards ("good")	a living document with a changelog	the bar, why it moved, when

A knowledge entry is small and linked:

```
---
claim: <one sentence that could be wrong>
sources: [<url or file actually read>]
confidence: high | medium | low
related: [<other entries>]
updated: <date>
---
<the evidence and the reasoning, in a few paragraphs>
```

Rules:

- **Decisions land where agents read, not where people chat.**
- **Link, don't pile.** Every entry points at the decisions, projects and evidence it depends on.
- **Prune.** Stale entries mislead agents with full confidence.
- **Keep private material out of shared space.** Shared context is shared.
- **Writing it down is part of the work,** not overhead after it.

Cross-team work stops depending on who you know. A marketing agent can read the product decision behind a feature; an engineering agent can read why the design went one way. Context compounds instead of being re-explained in every meeting.

---

10. Move the bar in the open

In agentic work "good" is supposed to move: what took a week is now an afternoon, so the afternoon version has to be better. The problem is never that the bar moves. It is whether everyone can see it moving.

- **A goalpost moves** when success is redefined after the work, privately. That teaches people to wait.
- **A bar rises** when capability changes and the change is visible and explained. That teaches people to look ahead.

Owners move the bar themselves:

- If good is not defined, **propose a definition.** Do not wait for one.

- If something can now be done better, **do it better and show what changed.**
- If the bar moved and you do not know why, **ask, then write the answer down.**

Keep the intent stable and the standards visible. Log every change to a standard with its reason.

---

11. Rituals

Per loop

1. Write or confirm the contract.
2. Run the fleet inside its bounds.
3. Review against the evidence bar.
4. Record what was learned: a note, a skill, a check.

Daily (15 minutes)

- What are today's intents? Do they still matter?
- What needs my review, in order? What can I kill?
- What did I learn that someone else, or some agent, should be able to query?

Weekly (30 minutes)

- Which standards moved, and are the changes written down?
- What recurred three times and should become a skill, a template or a check?
- Where did escalations pile up, and what bound or intent would have prevented them?
- Where did freed time go?

---

12. Anti-patterns

YOU NOTICE	WHAT IT USUALLY MEANS	DO INSTEAD
Lots of output, little that ships	attention is the bottleneck	lower the work-in-progress limit
"The agent did it"	ownership diffused	whoever delegated owns what ships
The same question asked again and again	the answer never landed in shared context	write it down where agents read
People wait for direction	initiative costs more than waiting, or intent was never given	check your own intent first, then lower the cost of proposing
A metric keeps improving, outcomes do not	the measure became a target	ask how big, from where, and whether the problem went away
Every result is "done"	no evidence bar	define done as an observable check
Reviews take longer than the work	verification was budgeted as a phase	budget review in proportion to delegated judgment
Nobody knows why the bar moved	a goalpost, not a rising bar	log standards changes with their reason

13. For leaders

What you owe: clear intent, real decision rights, an evidence bar for done, safety to try, standards that are visible when they move.

What you ask for: ownership of outcomes, proposals rather than status updates, an early signal when reality diverges from the plan, verification before claiming success, and what was learned written where others can use it.

How to read performance. No single metric measures value. Better questions: How big were the commitments? Where did they come from? Did the underlying problem actually go away? Did others work better because of this person? And before concluding someone lacks initiative, check what the environment taught them initiative costs.

Go first. When you feel a gap in someone, the first thing to check is your own intent.

14. Starter kit

Copy these into your workspace. They are also in the skill's `templates/` folder.

- `contract.md`: the five-term contract (section 3).
- `ask.md`: a decision request (section 7).
- `decision.md`: a decision record (what, why, alternatives, owner, date).
- `daily-review.md`: the daily ritual (section 11).

Agents expand what we can attempt. Agency decides what we attempt. Judgment decides what is true.

Own problems, not tickets. Doubt is a reason to start, not to wait. Sharp on the what and why, free on the how. Autonomy is a contract, and the leader signs first. Ration attention, not ambition. Your role is where you start, not where you stop.